

Automating the Management of Software Systems

Vishal Palaniappan

September 6, 2026

1 Introduction

Software systems are an essential part of modern society and when they fail to function as intended, the implications can be enormous. It is therefore vital that systems are built to minimize the possibility of failure and that, when failures do occur, effective diagnostic processes expedite the systems recovery and prevent the recurrence of failures.

To address this need, this document presents a framework called the Design Learning Platform (DLP) that fully automates the diagnosis of software failures. First, I will first explore the nature of the design on its own and explain how a closed semantic world can predict and resolve failures. Next, I will talk about how the design is applied to software systems and explain the diagnostic automation that it enables. Finally, I will talk about how this framework enables the complete automation of software system management and discuss how this is practically achieved by the platform.

2 The Design

A design is a closed semantic world. It contains participants whose state is transformed through the design's unambiguous behaviors. When the environment interacts with the design, it introduces a change to the state of the world. This state transition may enable new behaviors, which the design deterministically resolves in response to the updated state, producing further state changes. When resolving a state, the design may either exhibit behavior directly or deterministically select which behaviors are enabled as a reaction to the state. The design continues reacting to each resulting state transition until no further behavior is enabled and the world reaches a stable semantic state. This chain of resolutions constitutes the structure of the design itself and provides the environment a fully specified path through it.

The design is said to be in a semantically invalid state when one of its intentions can't be realized as a result of the state. When the environment provides an input into this closed world, in order to continue successfully realizing its intentions, the design must prevent semantically invalid states from persisting in its reality. To achieve this, semantic invariants act as markers which identify the semantically invalid state and predict the failure of downstream behaviors. This establishes an invariant path from the semantically invalid state to the intention that can't be realized and in order to restore semantic validity, the design must provide a semantically valid path to resolve the state.

Since the design is completely unambiguous and semantic in nature, another valid way to represent it as potential narratives within a closed semantic world and I will be using this framing in the rest of this document because it is more accessible. These narratives can be constructed by identifying the participants involved, the transformation being applied and the decisions that were made. When an environment provides an impulse into the design, a specific narrative is selected in this world. When looked at in this way, a semantically invalid path can be seen as a narrative that must be eliminated from the potential narratives of this closed semantic world and instead, the design must provide a semantically valid narrative for the environmental impulse as it moves through the design. By only allowing semantically valid narratives in the closed semantic world, the design eliminates failures with respect to all the known invariants.

In the first path, since the design does not provide a semantically valid narrative for the environment, the protected behavior is reached. The failure can be automatically debugged because the failure will be predicted by the violated invariant. However, in the second path, the design does provide a semantically

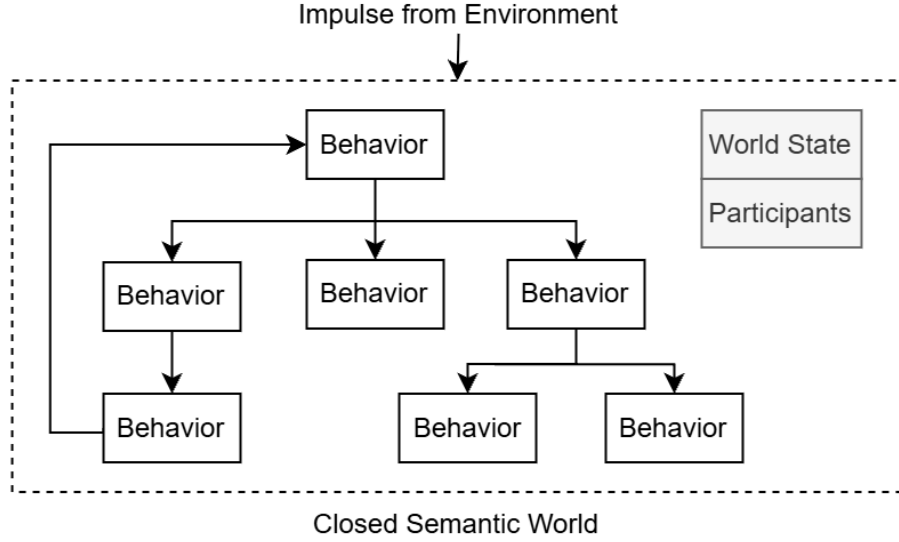


Figure 1: A block diagram of the design.

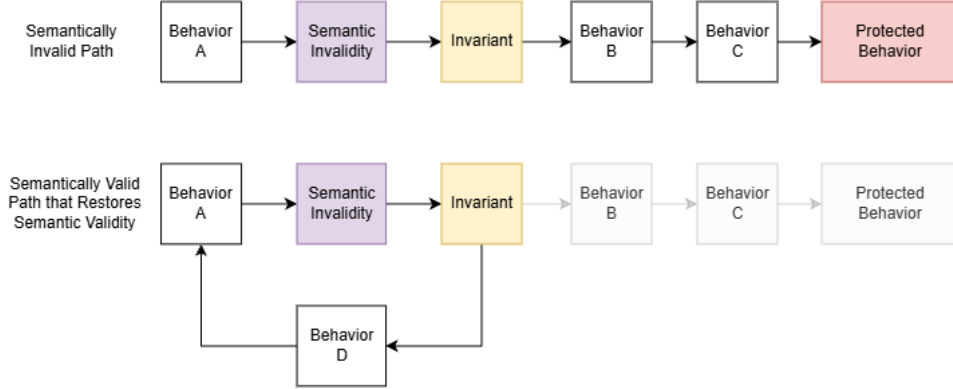


Figure 2: Invariant Path

valid narrative to restore semantic validity and the potential bug is eliminated. Using this structure, given a semantically invalid state, the invariant can be automatically tested by walking the path and identifying if the design provides a semantically valid narrative. In this sense, it is also fair to say that semantic invariants protect the downstream behavior because they ensure that the protected behavior can never be reached through the semantically invalid narrative.

Since a design is a closed semantic world, the invariants intrinsic to the design can be identified through the control flow and data dependencies within the design's meaning. As such, the design intrinsic invariants are identified by definition and the behavior of the design can be modified to ensure that it respects every invariant and is internally consistent. If a design is tested to ensure that it respects all its known invariants and it still fails, it means that the environment which caused the failure is revealing unknown semantics that the design must learn through root cause analysis. This automates failure diagnosis because there is no other interpretation of the failure. In this process, the design becomes progressively more intelligent as it learns new semantics and invariants so that it can continue realizing its intentions in its operational environment.

In this sense, through the invariants, the design absorbs the domain knowledge needed to prevent the failures within its closed semantic world. The design itself is reshaped by the invariants to only allow semantically valid narratives to persist. In this process, by working with the design, debugging, testing and failure diagnosis are automated. However, it is clear that the diagnostic automation isn't a result of complex

analysis of the world, instead, it is the result of the world being fully constructed, resulting in unambiguous diagnosis of failures. In this sense, every time ambiguity is eliminated in a meaningful way in the closed semantic world, a new form of automation will emerge.

3 Testing Code Block

```
participant("basket")

property(
  name="length"
  description="Number of books in basket"
  type="number"
)
property(
  name="type"
  description="Data type of participant (list, object, str etc)"
  type="string"
)
domain(
  contains="books"
)
dcs(
  # Domain structure for compression
)
getLength()
getType()
```